

Taller: análisis de tráfico LAN y predicción bajo congestión

Curso de Inteligencia Artificial — Introducción a Machine Learning

Propósito

Medir el comportamiento de una LAN en condiciones de carga controlada, construir una tabla de observaciones a partir de capturas y contadores de red, y evaluar si un modelo de regresión lineal permite anticipar la latencia. Identificar cuándo la congestión hace que la relación deje de ser aproximadamente lineal.

Resultados de aprendizaje

Al completar el taller, el equipo podrá: (1) capturar tráfico y contadores sin confundir bytes observados con capacidad total del enlace; (2) calcular tasas y métricas de calidad por ventanas de tiempo; (3) ajustar y evaluar un modelo de regresión; (4) reconocer señales de congestión y comunicar los límites de una predicción.

Escenario, equipos y reglas de medición

Trabajar en una LAN de laboratorio administrada o expresamente autorizada, con dos equipos propios en el mismo segmento o a través de un switch del laboratorio. El equipo A genera y captura tráfico; el equipo B recibe tráfico y responde a las sondas. Se recomienda un enlace cuya capacidad nominal se conozca y un generador de tráfico con límite configurado, por ejemplo `iperf3`. Capturar solo metadatos necesarios para la actividad: tiempos, tamaño, protocolo y conteos; evitar payloads, credenciales y datos de otras personas. Si la red es compartida, usar un enlace o VLAN aislado para la etapa de carga elevada y respetar los límites de operación definidos por el laboratorio.

Requisitos: Linux con `tcpdump`, `ip`, `ss` o `netstat`; `ping`; Python con `pandas`, `numpy`, `matplotlib` y `scikit-learn`. Para generar carga controlada, `iperf3` en A y B. Si `netstat` no está instalado, usar `ss` y los contadores de `ip -s link`. El análisis puede hacerse después en otra computadora con los CSV exportados.

1. Preparación y línea base (20 min)

1. Identifique las interfaces y la dirección del equipo B. Anote medio (Ethernet/Wi-Fi), velocidad nominal, topología, número aproximado de usuarios y hora. Use una interfaz de laboratorio conocida; no capture indiscriminadamente la interfaz `any`.

```
ip -br addr
ip -s link show dev eth0
ss -s
# Alternativa si esta disponible:
netstat -i
```

Sustituya `eth0` y `192.0.2.2` por la interfaz y dirección reales del laboratorio. `192.0.2.2` es un marcador de ejemplo.

2. Durante al menos cinco ventanas de 60 segundos sin carga experimental, mida RTT y pérdida entre A y B. Guarde salidas con sello de tiempo. Un comando por ventana:

```
date -Is
ping -n -i 1 -c 60 192.0.2.2 > ping_base_01.txt
```

3. Antes y después de cada ventana registre `ip -s link show dev eth0`. Calcule diferencias de bytes, paquetes, descartes y errores. Los contadores acumulados requieren resta entre marcas de tiempo; `netstat -i` también puede mostrar contadores, según sistema y versión.

2. Captura y escenarios de carga (40–60 min)

4. En B inicie `iperf3 -s`. En A, para cada escenario, capture metadatos IP con filtro hacia B. Limite la captura a encabezados y conserve el archivo únicamente en el laboratorio:

```
sudo timeout 65 tcpdump -i eth0 -n -s 96 \
  'host 192.0.2.2 and (tcp or udp or icmp)' \
  -w escenario_01.pcap
```

Inicie la captura unos segundos antes de la ventana efectiva y registre el inicio y fin de esos 60 segundos. El filtro observa tráfico entre A y B, no todo el tráfico del enlace. Si el objetivo es la carga del enlace completo, utilice contadores de interfaz y explique que pueden incluir tráfico ajeno o retransmisiones.

5. Genere carga en pasos crecientes, definidos por el docente según el enlace: por ejemplo objetivo de 0%, 20%, 40%, 60% y 80% de su capacidad *medida*, no una tasa universal. Repita cada nivel al menos tres veces y alterne el orden para reducir sesgos temporales. Para UDP, `-b` fija una tasa solicitada; no equivale a tasa efectivamente recibida:

```
iperf3 -c 192.0.2.2 -u -b 40M -t 60 --json \
  > iperf_escenario_01.json
```

Ajuste 40M al escenario y a la capacidad autorizada. No lleve la red compartida a saturación. Si se dispone de un enlace aislado, añada una prueba de carga alta para estudiar el cambio de régimen. Registre además RTT y pérdida con `ping` simultáneo a cada carga. Mantenga fijo el tamaño y tipo de tráfico en las repeticiones.

6. Al cierre de cada ventana vuelva a leer los contadores de interfaz y `ss -s` o `netstat -s`. Documente si hay pérdida, errores, descartes o retransmisiones y la herramienta que produjo cada cifra. Para una inspección básica de la captura:

```
tcpdump -nn -tt -r escenario_01.pcap | head
```

3. Construir el conjunto de datos (30 min)

Una fila debe corresponder a una ventana de medición válida; no use cada paquete como una observación independiente de latencia media. Cree `ventanas.csv` con estas columnas:

Columna	Definición
inicio, escenario, replica	Tiempo de inicio ISO 8601, nivel de carga y número de repetición.
duracion_s	Duración efectiva de la ventana, normalmente 60 s.
bytes_rx, bytes_tx	Diferencias de contadores de interfaz entre inicio y final, en bytes.
rtt_medio_ms	RTT medio de las sondas respondidas dentro de la ventana.
perdida_pct	Porcentaje de sondas sin respuesta en la ventana.
descartes, errores	Diferencias de contadores de interfaz; indique si son RX o TX.

Calcule $tasa_rx_mbps = 8 * bytes_rx / (duracion_s * 1e6)$ y análogamente TX. Si ambas direcciones comparten el mismo medio, analice también la suma, pero no la denomine “utilización del enlace” sin especificar capacidad efectiva y medio. Alinee inicio y fin de captura, sondas y contadores; descarte o etiquete ventanas incompletas. Evite contar encabezados de una PCAP truncada como si representaran todos los bytes reales de interfaz.

4. Exploración con Python (30 min)

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("ventanas.csv", parse_dates=["inicio"])
df["tasa_tx_mbps"] = 8 * df["bytes_tx"] / (df["duracion_s"] * 1e6)
print(df.info())
print(df.isna().sum())
print(df.groupby("escenario")[["tasa_tx_mbps", "rtt_medio_ms",
                                "perdida_pct"]].agg(["median", "min", "max"]))
fig, ax = plt.subplots(figsize=(7, 4))
ax.scatter(df["tasa_tx_mbps"], df["rtt_medio_ms"],
           c=df["perdida_pct"], cmap="viridis")
ax.set(xlabel="Tasa TX medida (Mbps)", ylabel="RTT medio (ms)")
fig.tight_layout(); plt.show()
```

Examine además RTT y pérdida contra tiempo, histogramas por nivel y diagrama de caja de RTT por escenario. Explique si aparecen una cola, dispersión creciente o un umbral a partir del cual aumenta la latencia.

5. Ajuste y evaluación de una regresión lineal (35 min)

Objetivo $y = rtt_medio_ms$; predictor inicial $x = tasa_tx_mbps$ medido. Ajuste $\hat{y} = \beta_0 + \beta_1 x$. Separe por *bloques de captura o sesiones* para que una repetición de la misma sesión no aparezca simultáneamente en entrenamiento y prueba. Si se pretende pronosticar condiciones futuras, use bloques posteriores para prueba. Asegure diversidad de cargas en entrenamiento y documente la franja de x que cubre la prueba. No seleccione el modelo usando la prueba repetidamente.

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Tras separar sesiones en dos tablas: train y test.
modelo = LinearRegression().fit(train[["tasa_tx_mbps"]],
                                train["rtt_medio_ms"])
real = test["rtt_medio_ms"].to_numpy()
pred = modelo.predict(test[["tasa_tx_mbps"]])
print("intercepto, pendiente:", modelo.intercept_, modelo.coef_[0])
print("MAE (ms):", mean_absolute_error(real, pred))
print("RMSE (ms):", np.sqrt(mean_squared_error(real, pred)))
print("R2:", r2_score(real, pred)) # requiere >= 2 ventanas de prueba
```

Incluya como referencia una predicción constante con la mediana de RTT del entrenamiento y compare MAE. Dibuje predicción frente a medición y residuos ($y - \hat{y}$) frente a carga. Reporte cuántas ventanas quedaron en cada partición.

6. Predicción de un escenario congestionado (25 min)

- Identifique una condición de *congestión observada* mediante varios indicadores: aumento persistente de RTT respecto de la línea base, pérdida o descartes, y caudal que deja de crecer proporcionalmente a la tasa ofrecida. Una tasa TX alta por sí sola no la prueba.
- Defina un nivel de carga alta autorizado y mida al menos tres ventanas nuevas en un enlace aislado. **Antes de medir su RTT**, pronostique con el modelo entrenado y registre $\hat{y}$, intervalo de cargas de entrenamiento y decisión de si es interpolación o extrapolación.
- Compare predicción con la mediana y el rango del RTT medido. Calcule error absoluto. ¿Los residuos crecen sistemáticamente en el régimen de carga alta? ¿El caudal recibido, la pérdida y los descartes respaldan la hipótesis de congestión?
- Si la red comienza a saturarse, la relación puede ser no lineal: limite el modelo lineal a su rango útil y proponga un modelo segmentado o una característica adicional para una actividad posterior. No presente una proyección lineal fuera de rango como garantía de operación.

Nota metodológica: para anticipar RTT a partir de carga que aún no ocurre, la tasa TX futura debe venir de un escenario planificado o de otro pronóstico; la regresión de este taller no predice por sí misma cuánto tráfico llegará.

Entregables y valoración (5,0 puntos)

Evidencia	Puntos
Protocolo reproducible: topología, interfaz, filtros, escenarios, repeticiones y control de captura	1,0
Tabla ventanas.csv , unidades, fuente de cada variable y controles de calidad	1,0
Análisis exploratorio y gráficos interpretados	1,0
Modelo, separación de sesiones, línea base y evaluación con residuos	1,0
Predicción registrada antes de la prueba congestionada, comparación y discusión de límites	1,0

Entregar informe de máximo cinco páginas, código ejecutable, **ventanas.csv** y un README con instrucciones y versiones de herramientas. No es necesario adjuntar PCAP sin anonimizar: bastan el protocolo, resúmenes y registros depurados. Señale qué mediciones fueron reales y qué hipótesis no pudieron comprobarse.

Preguntas de reflexión

- ¿Por qué observar más bytes en A no necesariamente permite conocer la utilización total del switch?
- ¿Qué diferencia hay entre RTT medido con **ping**, latencia unidireccional y tiempo de respuesta de una aplicación?
- ¿Por qué un MAE pequeño bajo carga normal no garantiza acierto durante congestión?
- ¿Cómo cambiaría el protocolo si los dispositivos se conectaran por Wi-Fi?